Fakultas Informatika
School of Computing
Telkom University

Aplikasi Berbasis Platform

Web Framework (3)

Session

- Session in Laravel has the same function with PHP native session, storing a variable on the server and this variable can be accessed and changed anywhere: Routing, Controller, or any page.
- Laravel's default session is managed by Laravel itself and is stored as a file. In addition to files, Sessions in Laravel can also be changed to be saved to the database or using other mechanisms.

Session (contd.)

- Laravel has two types of session: regular sessions and flash sessions.
- A regular session behave just like PHP native session, will always exist as long as it hasn't been deleted, timed out, or the browser is closed.
- While a flash session is a session that only applies to one request (one time only), after that Laravel will delete it automatically. So the flash session is suitable for displaying information.

Using Session

```
//set session
session()->put('email', $email);

//get session
$name = session('name');

//check session exist
if (session()->has('email'))
  return redirect('/product’);

//delete all session
session()->flush();
```

Middleware

- Middleware in Laravel is a mechanism to filter the request data. If some condition is fulfilled, then the request can be forwarded or redirected.
- The condition can be placed before the request is forwarded or after the request is received
- Example case:
  - we can use 'before' in authentication case: before a controller or view is called, we have to check whether the user has signed in or not
  - we can use 'after' in logging case: after we call the controller or view, we want to store the activity we've done into the database

Auth Middleware

- Laravel is bundled with a middleware for authentication, called Auth
- Like Session, we can use Auth for checking, authenticating (signing in), and signing out
- Auth redirects the login page to the route named 'login', so make sure you add the route name to your login page route

```
Route::get('/login', function () {
  return view('login');
})->name('login');
```

Auth Middleware (contd.)

- Auth uses ‘users’ table, so make sure you migrate …create_users_table.php migration file
- Auth uses ‘bcrypt’ hashing in password column for authenticating, so make sure your password data in users table has been ‘bcrypt’-hashed
- To bcrypt-hash a string, you can use function Hash::make(‘string you want to hash’) from library Illuminate\Support\Facades\Hash in controller, or function bcrypt(‘string you want to hash’) in blade view

Using Auth: checking

- Use Auth checking feature in route or controller

```
Route::get('/login', function () {
  if (Auth::check()) return redirect('/product');
  return view('login');
})->name('login');
```

- Use Auth checking feature for route

```
Route::resource('product', ProductController::class)->middleware('auth');
```

Using Auth: checking

- Use Auth checking feature in view

```
@auth
<div class="row justify-content-end" style="margin-top:2%">
  <div class="col-3">
 {{ Auth::user()->name }}
    <a href="/logout" class="btn btn-warning">Logout</a>
  </div>
</div>
@endauth
```

Using Auth: authenticating

- Use Auth authenticating feature in controller

```
...
use Illuminate\Support\Facades\Auth;

class SiteController extends Controller
{
  public function auth(Request $req) {
  if (Auth::attempt(['email'=>$req->em, 'password'=>$req->pwd])) {
  //jika ada nilai lain selain data user yang ingin disimpan di session, baru
gunakan session disini
  return redirect('/product');
  }
  return redirect('/login')->with('msg', 'Email / password salah');
  }
...
```

Using Auth: signing out

- Use Auth signing out feature in controller or route

```
Route::get('/logout', function () {
  Auth::logout();
  return redirect('/login');
});
```

Advancing Yourself

- Implement front-end templating, use Blade directive (see Practicum Book)
- Use form validation (see Practicum Book)
- Use other useful libraries:
  - Datatable: ease data output into datatable view
  - Storage: upload and download file
  - Email: email sending feature

API Routing

- When using Laravel, we can differentiate between web access or API access.
- API routing settings are in the routes/api.php file.
- The difference with web.php route:
  - the routes inside api.php (API routes) will be automatically prefixed with /api/
  - API routes has a throttling middleware that applied by default, so a client can request a route only x time per minute (preventing DoS attack)

API Controller

- Controller file for API is same for the web. The only difference is, the controller function for API doesn't return a view, but a JSON text

- Laravel provides the conversion to JSON, and with addition the HTTP return status

```
return response()->json($article, 200);
```

API Authentication

- Unlike web, authentication via API usually uses token mechanism
- Token will be attached in the request header
- Laravel provides two optional packages:
  - Passport
  - Sanctum
- Passport uses OAuth2 token provider
- Sanctum can use both cookie-based authentication (must have same top-level domain) or bearer token

Using Sanctum: Configuration

- Laravel 8 or below
  - Install via Composer:

```
composer require laravel/sanctum
```

  - Create Sanctum configuration and migration file:

```
php artisan vendor:publish --provider="Laravel\Sanctum\SanctumServiceProvider"
```

  - Create table to store Sanctum token:

```
php artisan migrate
```

- Laravel 9 or 10: Sanctum already included in installation
- Laravel 11
  - Install and configuration in one command:

```
php artisan install:api
```

Using Sanctum: Protecting Routes

- In routes/api.php, group all routes that protected by Sanctum (we must use Sanctum token to access those routes)

```
Route::group(['middleware' => ['auth:sanctum']], function () {
   …
});
```

Using Sanctum: Creating token

- When another application (ex: mobile app) wants to make an API request to a route that protected by Sanctum, that application must use a Sanctum token
- So we must create a route or controller function that accepts the user's email / username, password, and device name (additional, any value), then exchanges those credentials for a new Sanctum token:

https://laravel.com/docs/11.x/sanctum#issuing-mobile-api-tokens

Using Sanctum: Using token

- Usually this creating token function only called once, for example when you login to the mobile app or when mobile app just access the API for the first time
- Because, for efficiency, the resulted token will be stored on the mobile app. So when mobile app wants to make another API request, it can just use the stored token and doesn’t have to call the creating token function again
- To use this resulted Sanctum token, mobile app must always attach the resulted token in the Authorization header as a Bearer token when it wants to access a Sanctum-protected route

Any question?

References

- https://laravel.com/docs/

Fakultas Informatika
School of Computing
Telkom University

THANK YOU